Hands-on Labs

Afternoon Session

Day 4 - Afternoon Labs

KavinSchool

Lab 4.3: Build an Evaluation Harness

Ethics, Bias Mitigation & Monitoring

KavinSchool

Lab 4.3: Evaluation Script

Goal: Build a script that runs test prompts, compares outputs to expected results, and logs pass/fail.

Requirements:

1. Define 10 test cases with expected keywords and forbidden terms
2. Run each test case through the LLM
3. Score each response (keyword matching)
4. Log results with pass/fail status
5. Print a summary: total pass rate, worst performers, average score

Stretch goals:

- Add LLM-as-judge scoring alongside keyword scoring

Capstone Project

Build Your Own AI-Powered Developer Assistant

KavinSchool

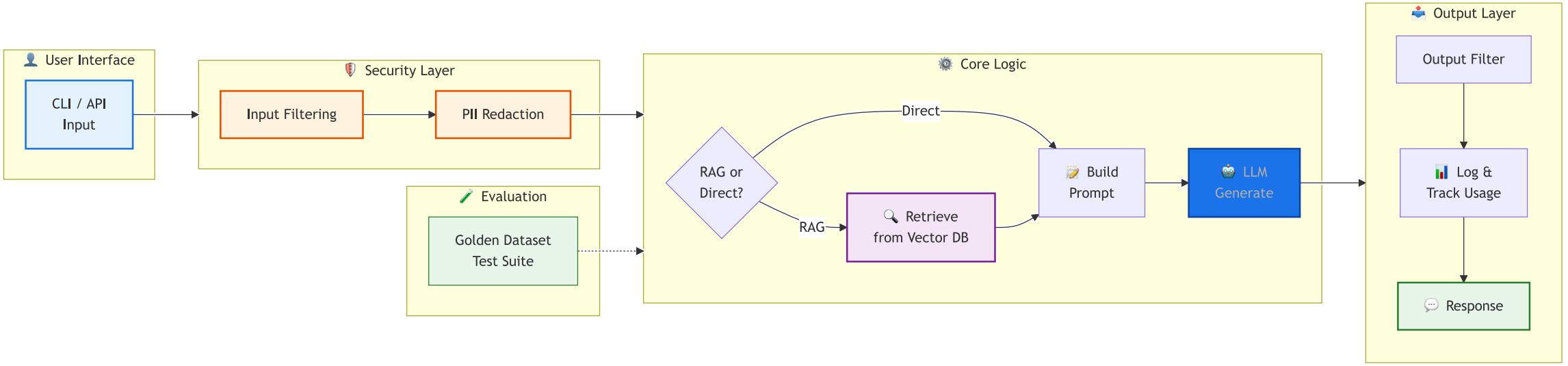
Capstone: Choose Your Project

Option	Description	Key Skills
A	Code Review Assistant	RAG over coding standards + prompt engineering
B	Documentation Q&A Bot	RAG + conversation context + source citation
C	Bug-Report Triager	Classification + structured output + eval

All options must demonstrate:

1. Prompt engineering (system message, structured output)
2. Conversation context OR RAG
3. Error handling (API failures, invalid input)
4. Basic evaluation (at least 5 test cases)

Capstone Architecture



- **Core Logic** - Combine robust prompt engineering with dynamic context (like RAG or chat history).
- **Safety & Resiliency** - Implement guardrails and error handling to manage invalid inputs and API failures.
- **Evaluation** - Continuously test your implementation against a curated set of evaluation cases to ensure quality.

Option A: Code Review Assistant

What it does: Reviews Python code against team coding standards.

```
# System prompt sketch
"""You are a code reviewer. Review the provided code against
these standards retrieved from the team's coding guide:
{retrieved_standards}

For each issue found:
1. Quote the problematic line
2. Explain why it's a problem
3. Suggest a fix

Rate the overall code quality: PASS / NEEDS_CHANGES / FAIL"""
```

Option A: Code Review Assistant

Components:

- RAG over coding standards documents
- Structured output (JSON with issues list)
- Input: paste code, get review

Option B: Documentation Q&A Bot

What it does: Answers questions about project documentation with source citations.

```
# System prompt sketch
"""You are a documentation assistant for the Python project.
Answer questions based ONLY on the retrieved documentation.
Always cite sources using [Source: filename].
If the docs don't cover the topic, say so clearly.
Remember context from earlier in the conversation."""
```

Option B: Documentation Q&A Bot

Components:

- RAG over project docs (markdown files)
- Multi-turn conversation history
- Source citation in every answer

Option C: Bug-Report Triager

What it does: Classifies bug reports by severity and routes them.

```
# System prompt sketch
"""You are a bug report triager. For each bug report:
1. Classify severity: CRITICAL / HIGH / MEDIUM / LOW
2. Categorize: UI / API / Database / Security / Performance / Other
3. Suggest which team should handle it
4. Extract key symptoms

Respond in JSON format:
{"severity": "...", "category": "...", "team": "...",
 "symptoms": [...], "summary": "..."}"""
```

Option C: Bug-Report Triager

Components:

- Structured JSON output
- Few-shot examples for consistent classification
- Evaluation against labeled test cases

Capstone: Getting Started

Step 1 (5 min): Choose your option (A, B, or C)

Step 2 (10 min): Design your system message and plan your architecture

Step 3 (30 min): Implement the core functionality

Step 4 (10 min): Add error handling and at least 5 test cases

Step 5 (5 min): Prepare a brief demo (2-3 example interactions)

File: `gen-ai-companion/day4/afternoon-labs/start/capstone.py`

> Start with the simplest version that works. Add features if time permits.

Capstone: Evaluation Criteria

Criterion	Points	What We're Looking For
Prompt engineering	25	Clear system message, appropriate techniques
RAG or context	25	Working retrieval OR multi-turn memory
Error handling	20	Graceful API failures, input validation
Evaluation	15	At least 5 test cases with pass/fail
Code quality	15	Clean, readable, well-structured

> This is not graded! The rubric helps you self-assess completeness.

Demo Time

Each learner gives a 5-minute demo:

1. Show your system message and explain your design choices
2. Run 2-3 example interactions
3. Show one error handling scenario
4. Share one thing you learned or would do differently

> The goal is to share and learn from each other, not to present a polished product.